



Machine-Level Programming IV: Jumps, Loops, and Functions

Prof. Yanjing Li



Today

- Jumps
- Loops
- Functions



Jumps: Instructions for Flow Control

- Instructions are executed sequentially unless there is a flow control instruction
- **Flow control** instructions allow the machine to execute a different part of code
 - Why do flow control?
 - To support if, while/for, etc.
- **Control flow** using condition codes + Jump command

Jumping

■ jX Instructions

- Jump to different part of code depending on condition codes

■ jmp: always jump

jX	Condition	Description
jmp	1 <i>always jump</i>	Unconditional
je	ZF <i>if the ZE</i>	Equal / Zero
jne	$\sim$ ZF <i>if not ZE</i>	Not Equal / Not Zero
js	SF	Negative
jns	$\sim$ SF	Nonnegative
jg	$\sim (SF \wedge OF) \ \& \ \sim ZF$	Greater (Signed)
jge	$\sim (SF \wedge OF)$	Greater or Equal (Signed)
jl	$(SF \wedge OF)$	Less (Signed)
jle	$(SF \wedge OF) \mid ZF$	Less or Equal (Signed)
ja	$\sim CF \ \& \ \sim ZF$	Above (unsigned)
jb	CF	Below (unsigned)



Conditional Branch Example (jX)

jX	Condition	Description
j1	(SF^OF)	Less (Signed)

```
long absdiff
(long x, long y)
{
    long result;
    if (x >= y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

absdiff:

```
    cmpq    %rsi, %rdi    # x:y
    j1      .L4           # jmp if x<y
    movq    %rdi, %rax
    subq    %rsi, %rax
    ret

.L4:      # x < y
    movq    %rsi, %rax
    subq    %rdi, %rax
    ret
```

Handwritten notes:

- if the j1 flag, go to L4
- Store it in control flags > x-y
- rax is for return value

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value



Conditional Branch Example (jX)

jX	Condition	Description
j1	(SF^OF)	Less (Signed)

```
long absdiff
(long x, long y)
{
    long result;
    if (x >= y)
        result = x-y;
    else
        result = y-x;
    return result;
}
```

```
absdiff:
    cmpq    %rsi, %rdi    # x:y
    j1      .L4            # jmp if x<y
    movq    %rax, %rdi
    subq    %rsi, %rdi
    ret

.L4:
    movq    %rsi, %rax
    subq    %rdi, %rax
    ret
```

cmpq sets the CC
j1 Line
Jump to Line if a < b

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value



Today

- Jumps
- **Loops**
- Functions

While Loop Example

C Code

```
long pcount_while  
(unsigned long x) {  
    long result = 0;  
    while (x) {  
        result += x & 0x1;  
        x >>= 1;  
    }  
    return result;  
}
```

we jump back
while we loop

- Count number of 1's in argument **x**
- Use conditional branch to either continue looping or to exit loop



While Loop Example Assembly

```
long pcount_while
(unsigned long x) {
    long result = 0;
    while (x) {
        result += x & 0x1;
        x >>= 1;
    }
    return result;
}
```

Register	Use(s)
%rdi	Argument x
%rax	result
%rdx	tmp

Checks the 0 flag

```
movq    $0, %rax    # result = 0
cmpq    $0, %rdi
je      .done
.L2:
movq    %rdi, %rdx
andq    $1, %rdx    # tmp = x & 0x1
addq    %rdx, %rax   # result += tmp
shrq    %rdi         # x >>= 1
jne     .L2          # if (x) goto loop
.done:
ret
```

shrq sets ZF if x is 0
Jump if ZF is 0

shift right

if the 0 flag is not set

For Loop Example

General Form

```
for ( Init; Test; Update )  
    Body
```

```
long pcount_for (unsigned long x)  
{  
    size_t i;  
    long result = 0;  
    for (i = 0; i < 64; i++)  
    {  
        unsigned bit = (x >> i) & 0x1;  
        result += bit;  
    }  
    return result;  
}
```

Init Test Update

i = 0

i < 64

i++

Body

```
unsigned bit =  
    (x >> i) & 0x1;  
result += bit;
```



For Loop Example Assembly

```
long pcount_for (unsigned long x)
{
    size_t i;
    long result = 0;
    for (i = 0; i < 64; i++)
    {
        unsigned bit = (x >> i) & 0x1;
        result += bit;
    }
    return result;
}
```

Register	Use(s)
%rdi	Argument x
%rax	result
%rdx	bit, tmp
%rcx	i

```
movq    $0, %rax    # result = 0
```

```
movq    $0, %rcx    # i = 0
```

Init loop variable

.L2:

```
    # loop:
```

```
movq    %rdi, %rdx    # tmp = x
```

```
shrq    %rcx, %rdx    # tmp = tmp >> i
```

```
andq    $1, %rdx    # bit = tmp & 0x1
```

```
addq    %rdx, %rax    # result += bit
```

```
incq    %rcx    # i++
```

```
cmpq    $64, %rcx
```

```
j1      .L2    # if(i<64) goto loop
```

```
ret
```

Loop body

Update loop variable

Test loop variable

.L2

%rcx - \$64

increment

←



Today

- Jumps
- Loops
- **Functions (aka. Procedures)**



Mechanisms in Procedures

■ Passing control

- To beginning of procedure code
- Back to return point

we need
to come back
to the
same place
after you've
run the function

■ Passing data

- Procedure arguments
- Return value

■ Memory management

- Allocate during procedure execution
- Deallocate upon return

↓
function stack
for regional memory

```
P (...) {  
  •  
  •  
  y = Q(x);  
  print(y)  
  •  
}
```

```
int Q(int i)  
{  
  int t = 3*i;  
  int v[10];  
  •  
  •  
  return v[t];  
}
```

Mechanisms in Procedures

■ Passing control

- To beginning of procedure code

- Back to return point

■ Passing data

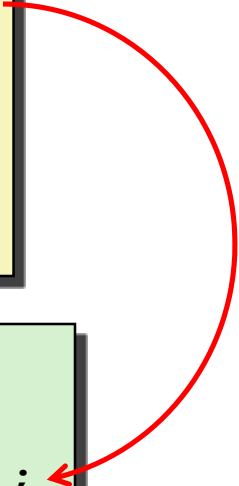
- Procedure arguments
- Return value

■ Memory management

- Allocate during procedure execution
- Deallocate upon return

```
P (...) {  
    •  
    •  
    y = Q(x);  
    print(y)  
    •  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    •  
    •  
    return v[t];  
}
```



Mechanisms in Procedures

■ Passing control

- To beginning of procedure code
- Back to return point

■ Passing data

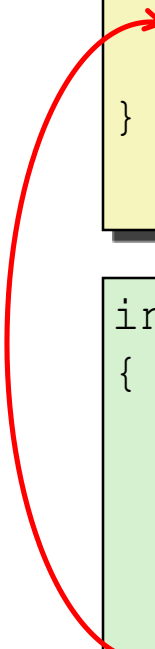
- Procedure arguments
- Return value

■ Memory management

- Allocate during procedure execution
- Deallocate upon return

```
P (...) {  
    •  
    •  
    y = Q(x);  
    print(y)  
    •  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    •  
    •  
    return v[t];  
}
```



Mechanisms in Procedures

■ Passing control

- To beginning of procedure code
- Back to return point

■ Passing data

- Procedure arguments
- Return value

■ Memory management

- Allocate during procedure execution
- Deallocate upon return

```
P (...) {  
    •  
    •  
    y = Q(x);  
    print(y)  
    •  
}
```



```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    •  
    •  
    return v[t];  
}
```


Mechanisms in Procedures

■ Passing control

- To beginning of procedure code
- Back to return point

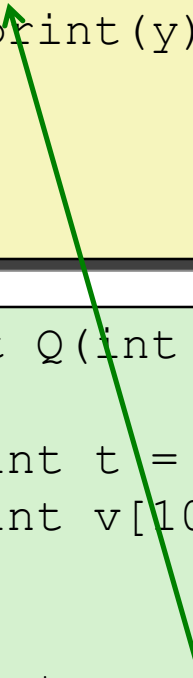
■ Passing data

- Procedure arguments
- Return value

■ Memory management

- Allocate during procedure execution
- Deallocate upon return

```
P (...) {  
    •  
    •  
    y = Q(x);  
    print(y)  
    •  
}
```



```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    •  
    •  
    return v[t];  
}
```

Mechanisms in Procedures

■ Passing control

- To beginning of procedure code
- Back to return point

■ Passing data

- Procedure arguments
- Return value

■ Memory management

- Allocate during procedure execution
- Deallocate upon return

```
P (...) {  
    •  
    •  
    y = Q(x);  
    print(y)  
    •  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    •  
    •  
    return v[t];  
}
```

Mechanisms in Procedures

■ Passing control

- To beginning of procedure code
- Back to return point

■ Passing data

- Procedure arguments
- Return value

■ Memory management

- Allocate during procedure execution
- Deallocate upon return

Accomplished using stack+ special instructions + conventions

Recursions are just like function calls

```
P (...) {  
    •  
    •  
    y = Q(x);  
    print(y)  
    •  
}
```

```
int Q(int i)  
{  
    int t = 3*i;  
    int v[10];  
    •  
    •  
    return v[t];  
}
```



Today

■ Procedures

- Stack Structure
- Calling Conventions
 - Passing control
 - Passing data
 - Managing local data

Where is the Stack?



x86-64 Linux Memory Layout

not drawn to scale

■ Stack

- Runtime stack (8MB limit)
- E. g., local variables

■ Heap

- Dynamically allocated as needed
- When call `malloc()`, `calloc()`, `new()`

■ Data

- Statically allocated data
- E.g., global vars, `static` vars, string constants

■ Text / Shared Libraries

- Executable machine instructions
- Read-only

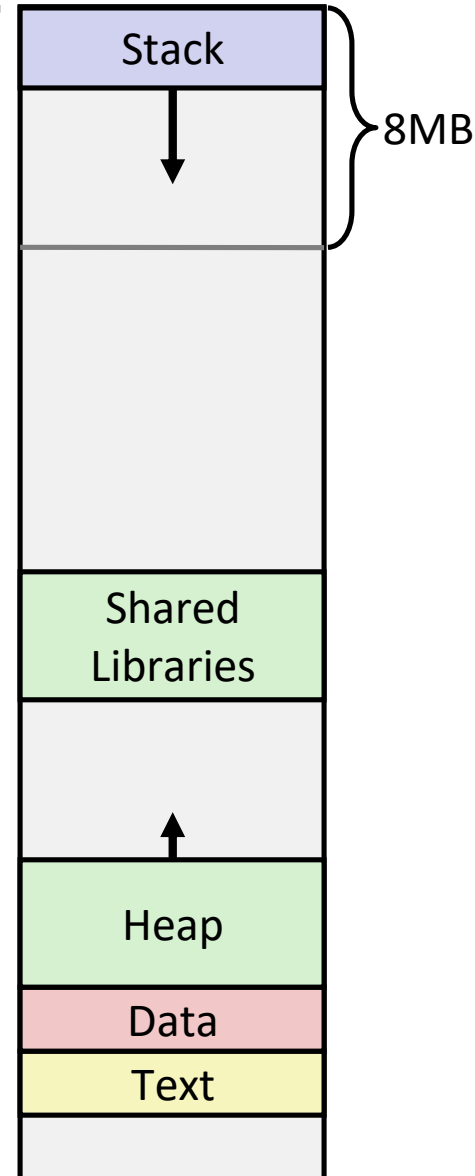
00007FFFFFFF

*top of
the stack
see's what's
running*

Hex Address



400000
000000



x86-64 Stack

- Grows toward lower addresses
- Register `%rsp` contains lowest stack address
 - address of “top” element

Actually, the top of the stack is at the lower address

Stack Pointer: `%rsp` →

Stack “Bottom”



Increasing
Addresses

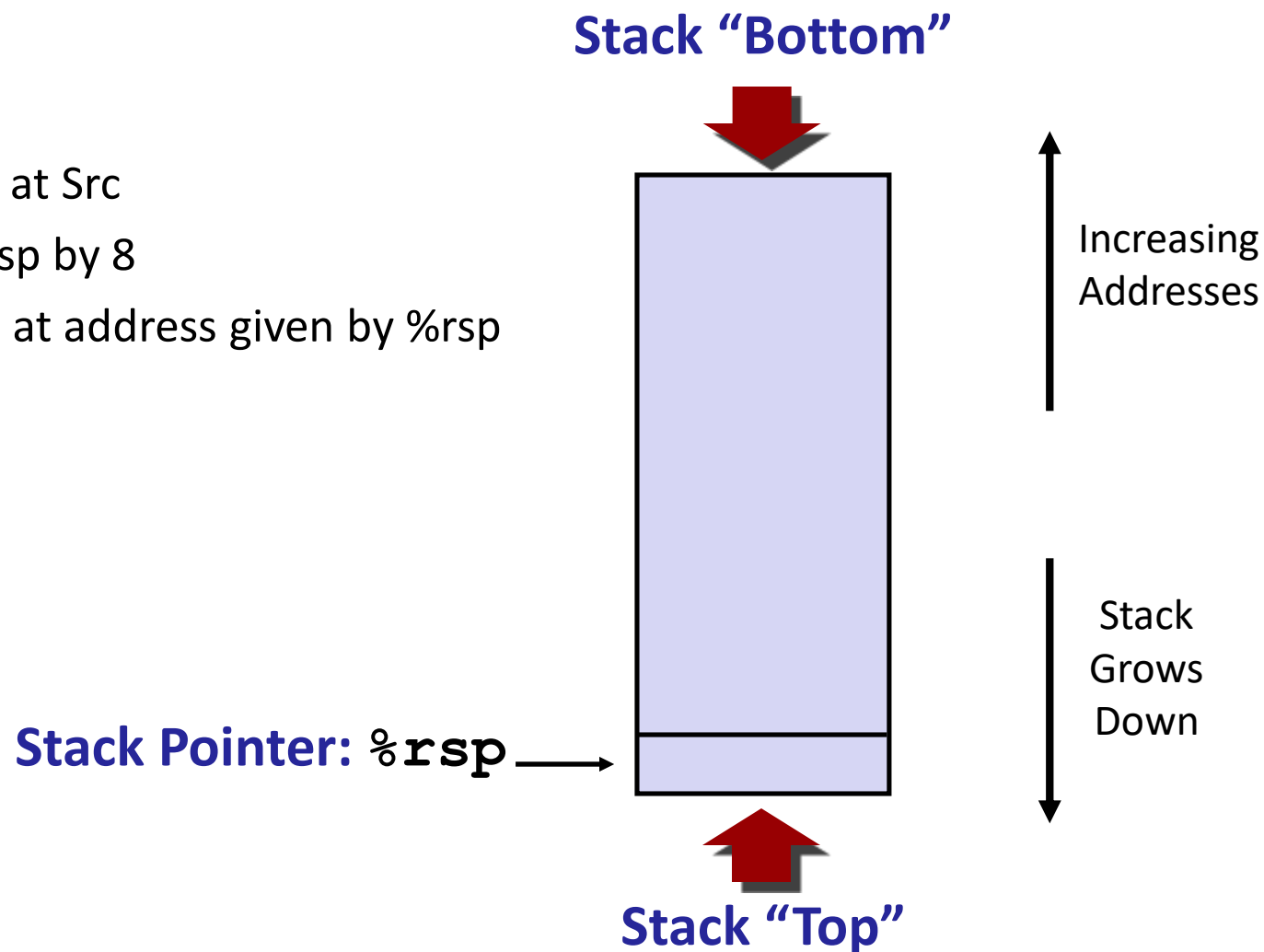
Stack
Grows
Down

Stack “Top”

x86-64 Stack: Push

■ `pushq Src`

- Fetch operand at Src
- Decrement `%rsp` by 8
- Write operand at address given by `%rsp`



x86-64 Stack: Push

■ `pushq Src`

- Fetch operand at Src
- Decrement `%rsp` by 8
- Write operand at address given by `%rsp`
- Example: `pushq %rbx`

move value from rbx to the stack

`%rbx` `0xFF00`

Stack Pointer: `%rsp` →

Stack “Bottom”



Increasing
Addresses

Stack
Grows
Down

Stack “Top”

x86-64 Stack: Push

■ `pushq Src`

- Fetch operand at `Src`
- Decrement `%rsp` by 8
- Write operand at address given by `%rsp`
- Example: `pushq %rbx`

`%rbx`

0xFF00

Stack Pointer: `%rsp`



Stack "Top"

Stack "Bottom"



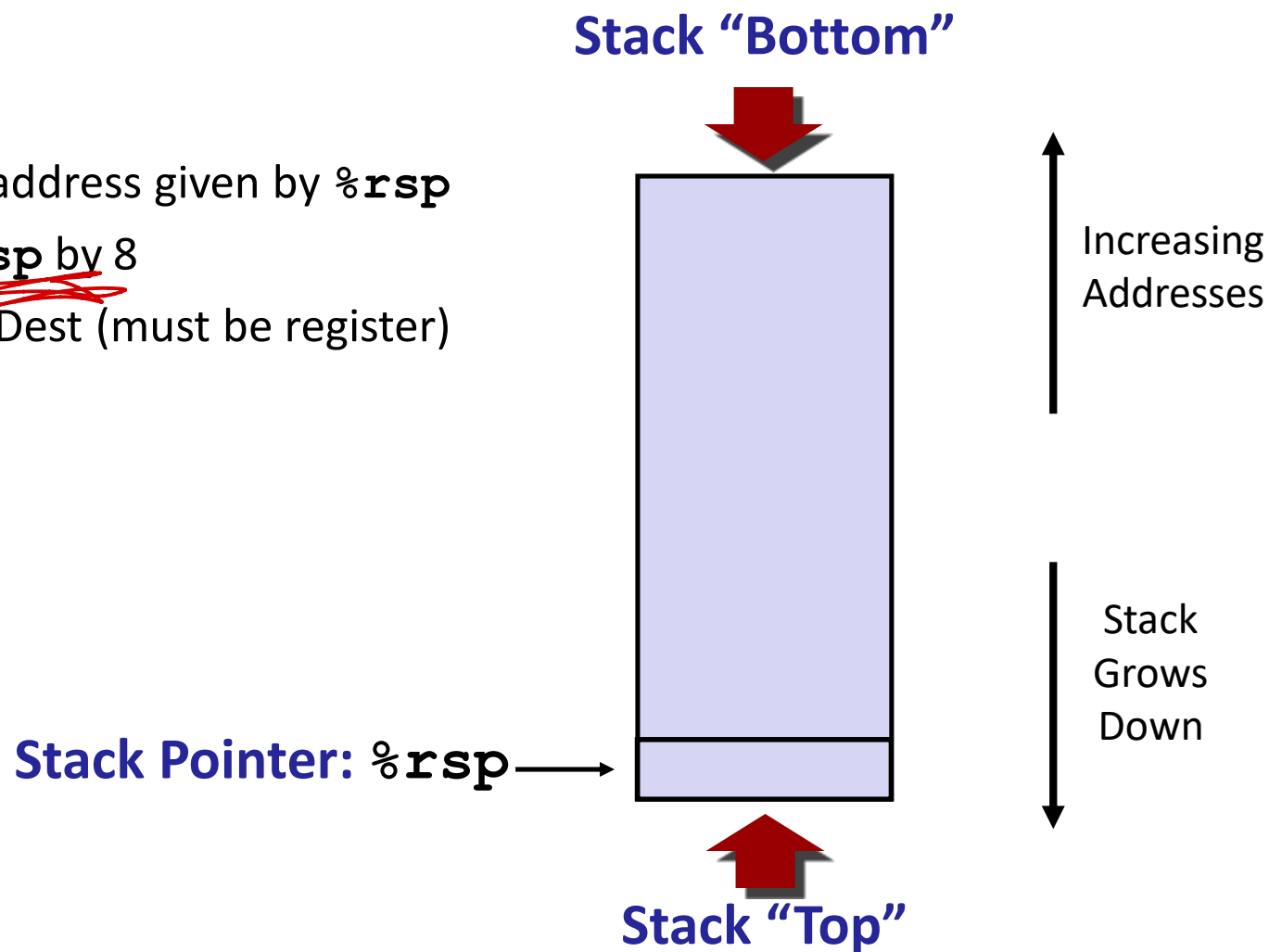
Increasing
Addresses

Stack
Grows
Down

x86-64 Stack: Pop

■ `popq Dest`

- Read value at address given by `%rsp`
- Increment `%rsp` by 8
- Store value at `Dest` (must be register)



x86-64 Stack: Pop

■ **popq Dest**

- Read value at address given by **%rsp**
- Increment **%rsp** by 8
- Store value at Dest (must be register)
- Example: **popq %rbx**

*When you pop
the data
out and adjust
the pointer, you're
essentially marking the
stack marker*

%rbx



Stack Pointer: %rsp →

0xFF00

Stack "Bottom"



↑
Increasing
Addresses

↓
Stack
Grows
Down

Stack "Top"



*push and pop
adjust the pointer*

x86-64 Stack: Pop

■ `popq Dest`

- Read value at address given by `%rsp`
- Increment `%rsp` by 8
- Store value at `Dest` (must be register)
- Example: `popq %rbx`

`%rbx`

0xFF00

Stack Pointer: `%rsp`

+8

Stack “Bottom”



Increasing
Addresses

Stack
Grows
Down

Stack “Top”

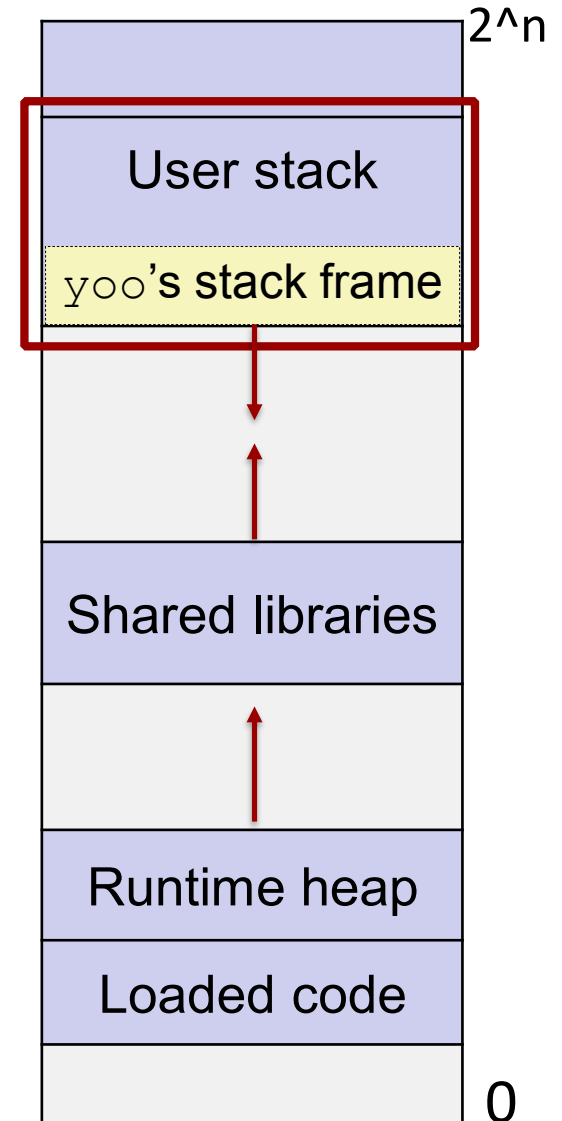


Stack and Procedure Calls

```
yoo (...)  
{  
  .  
  .  
  .  
  .  
}
```

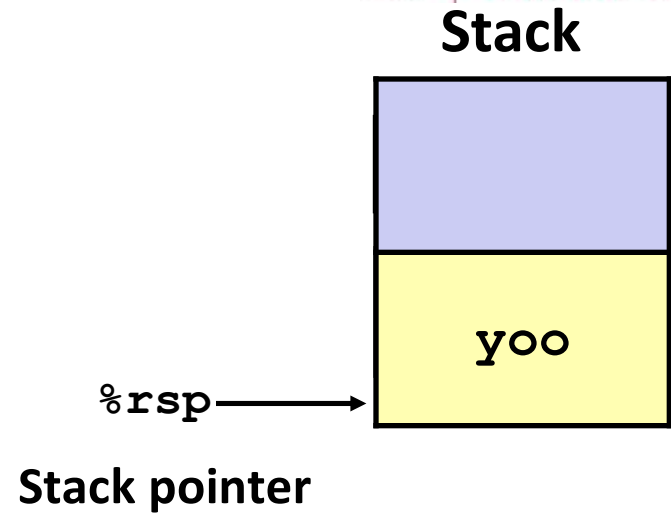
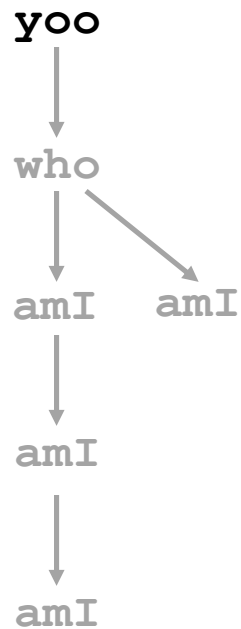
*The last function
you call will
return first*

Memory address space

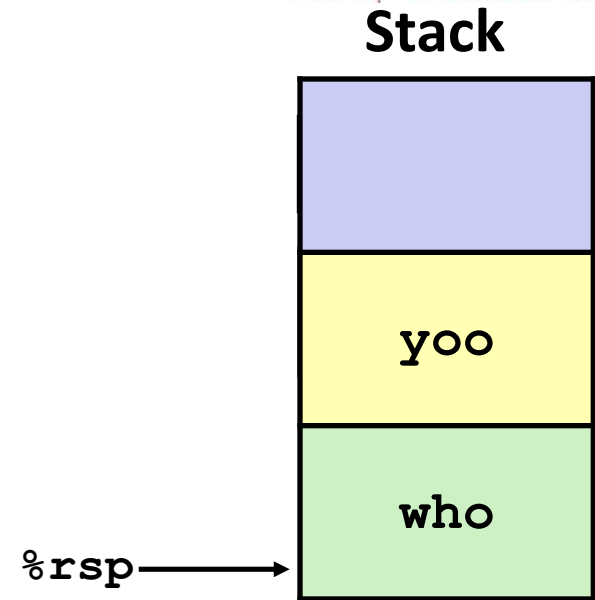
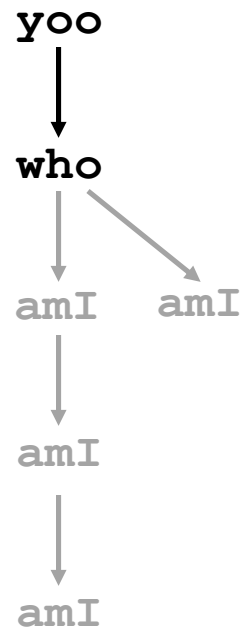
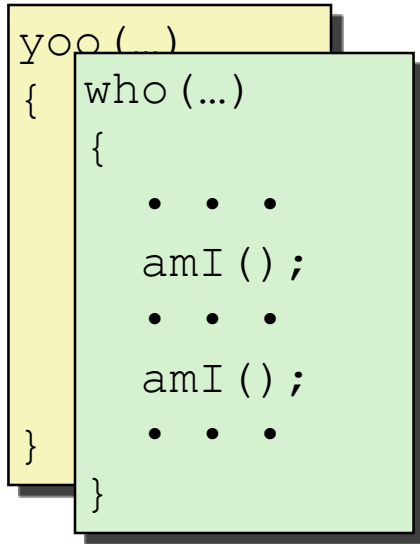


Example

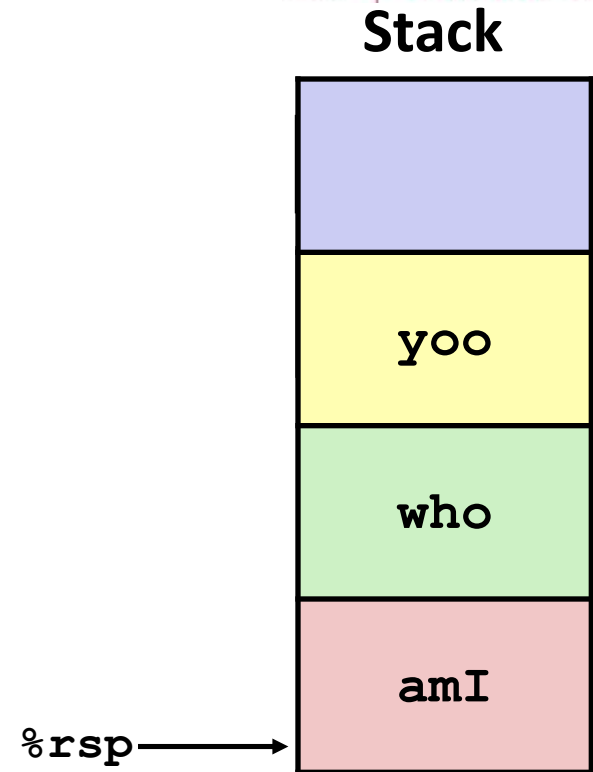
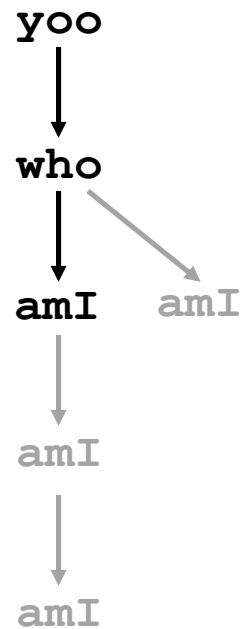
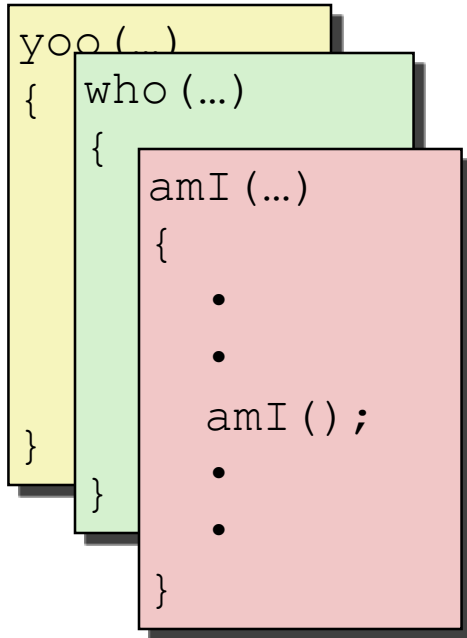
```
yoo (...)  
{  
  .  
  .  
  who ();  
  .  
  .  
}
```



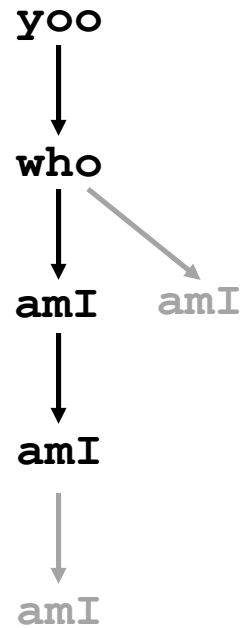
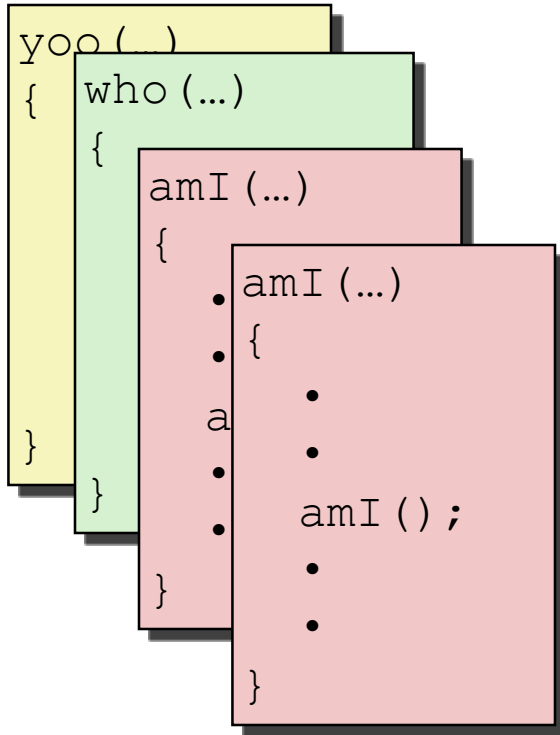
Example



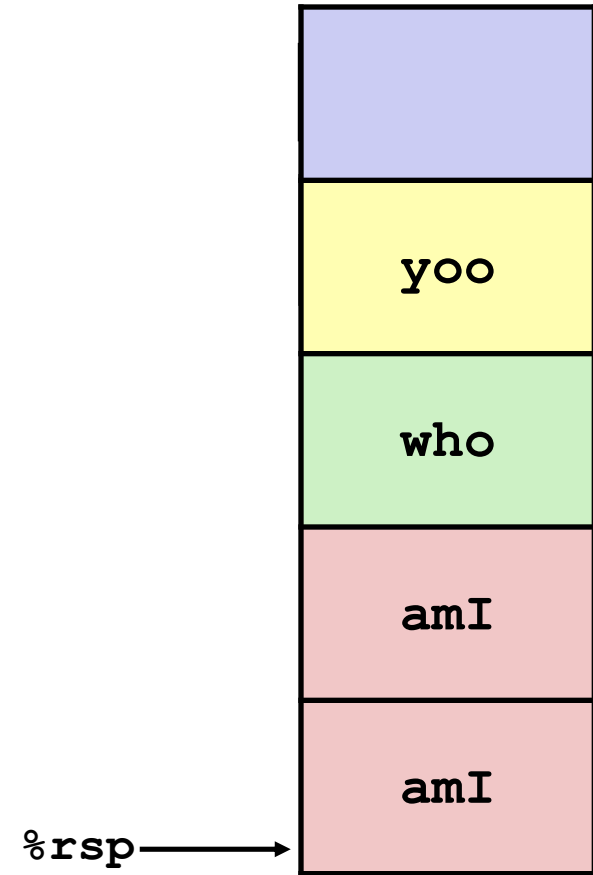
Example



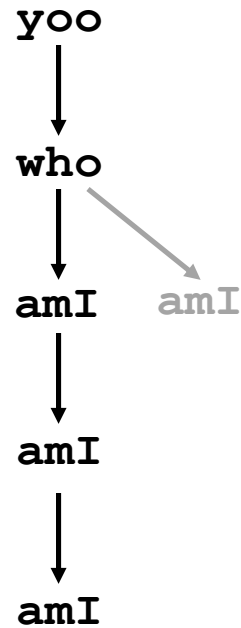
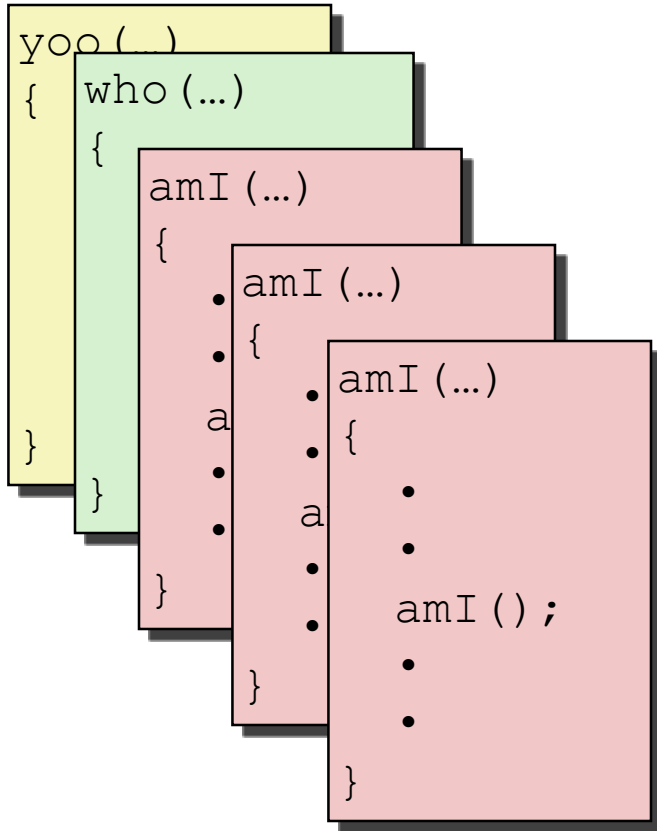
Example



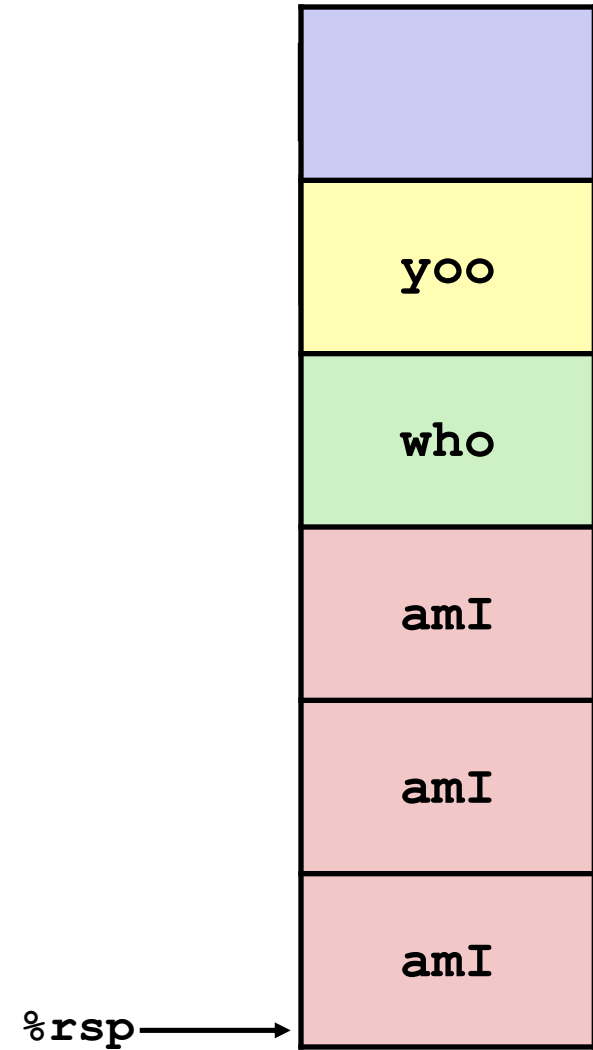
Stack



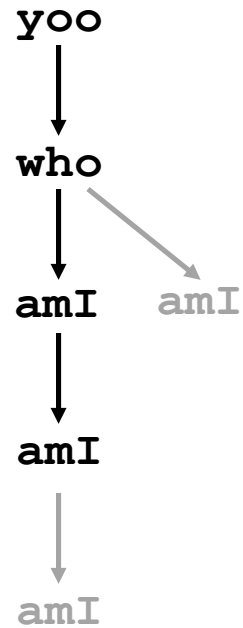
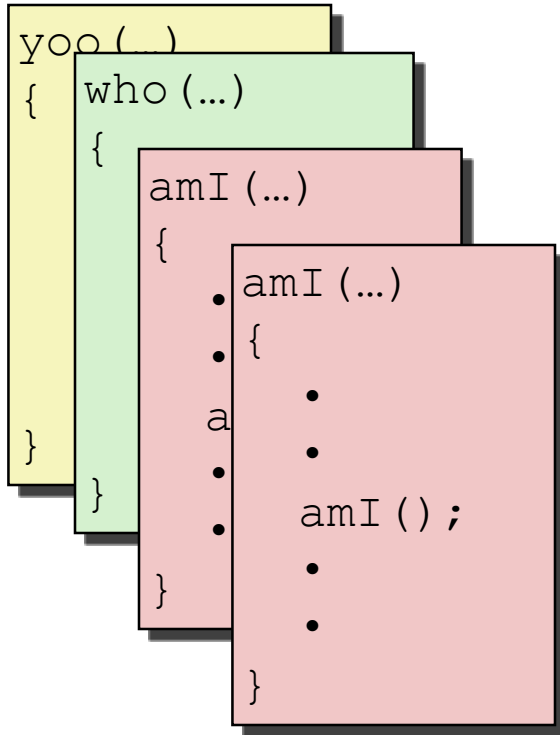
Example



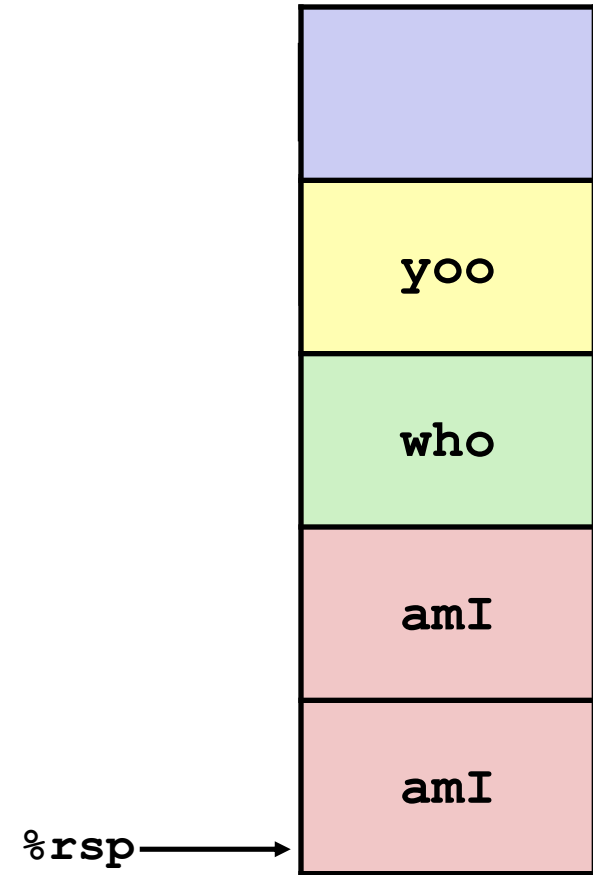
Stack



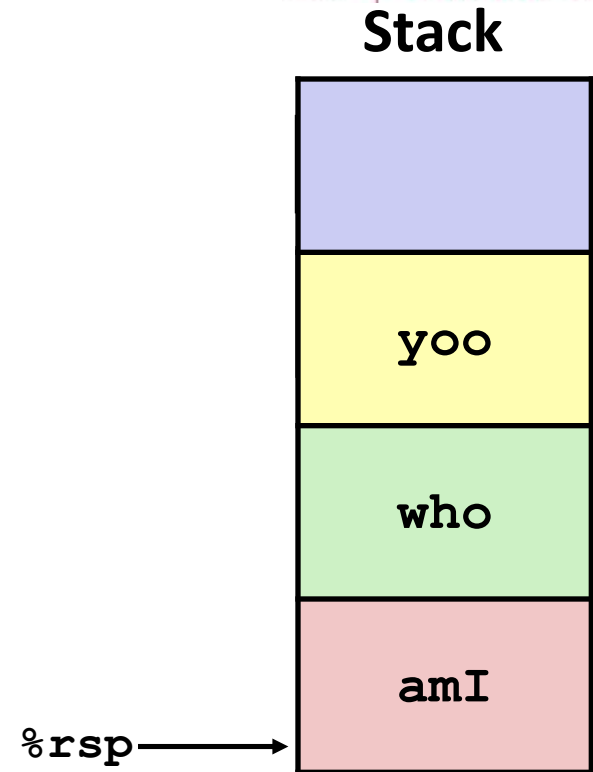
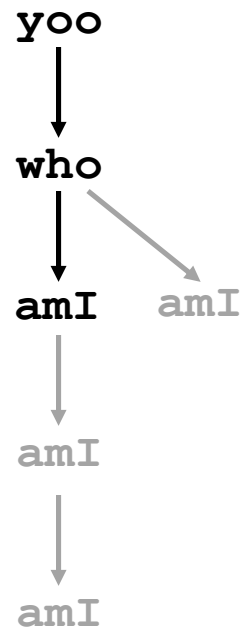
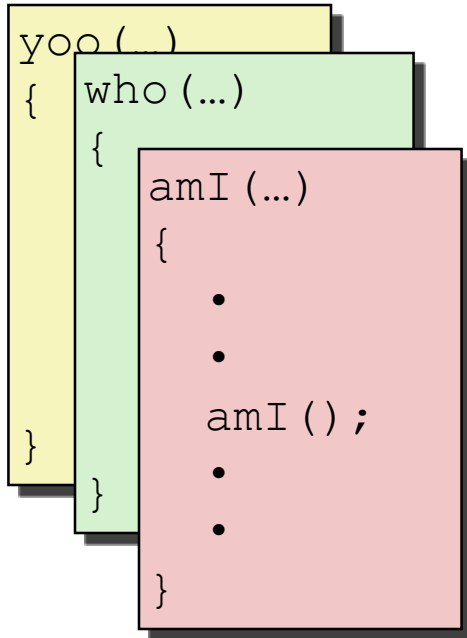
Example



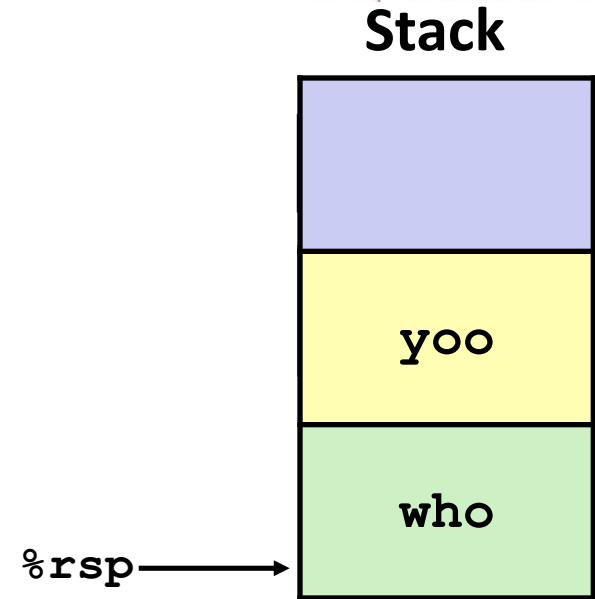
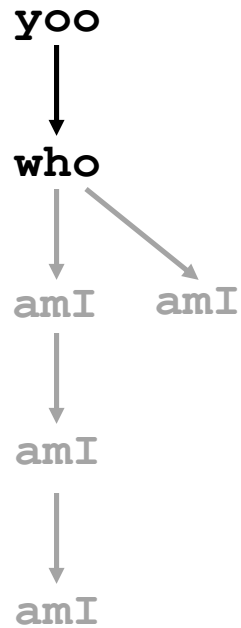
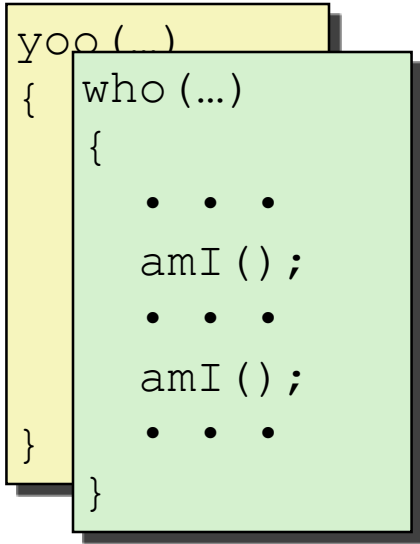
Stack



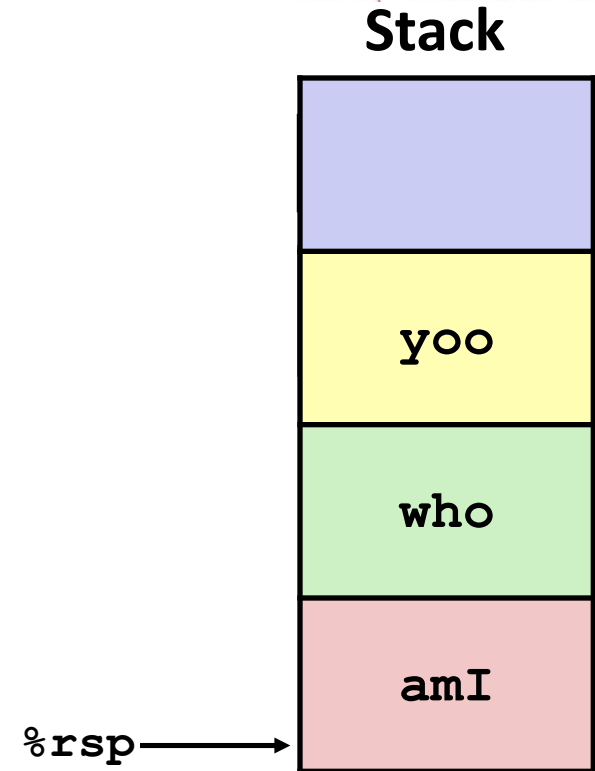
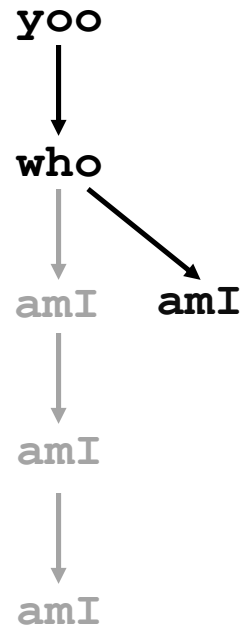
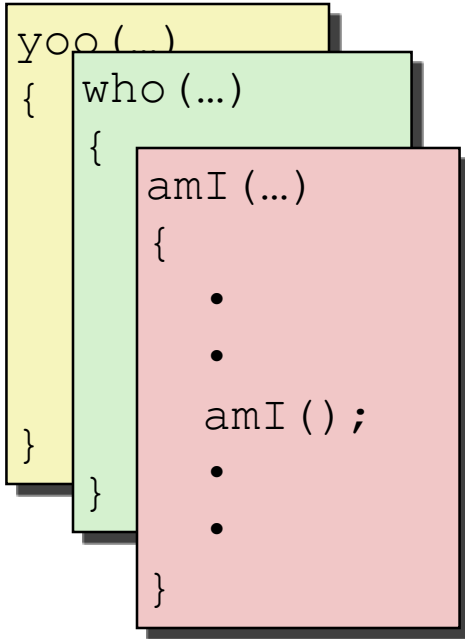
Example



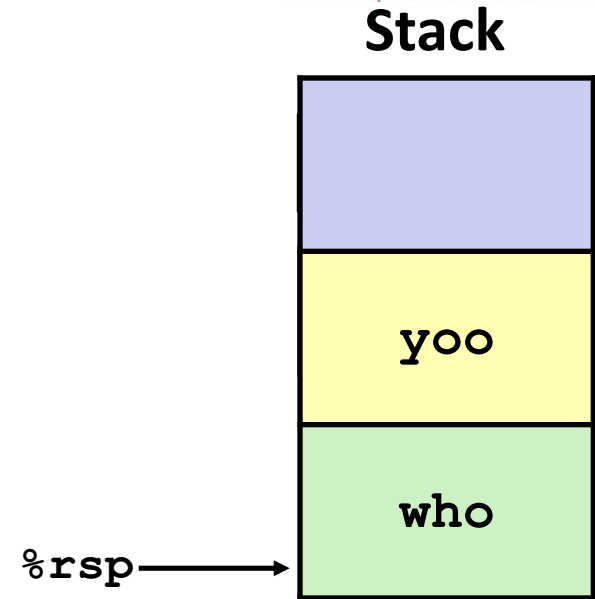
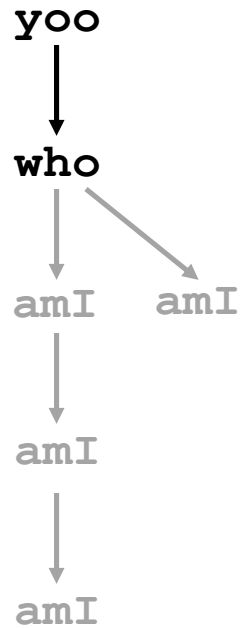
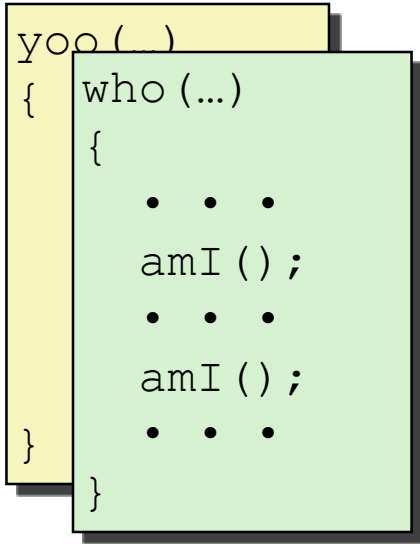
Example



Example

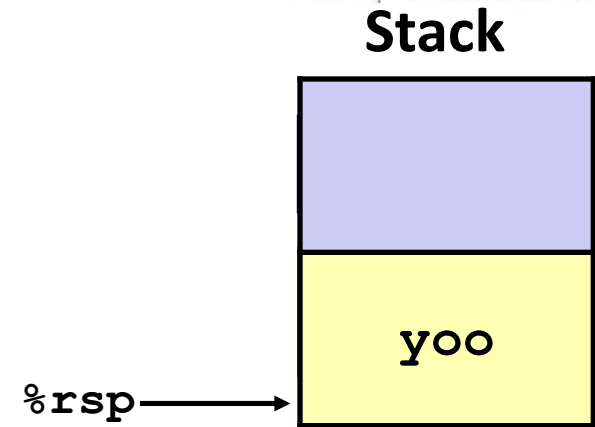
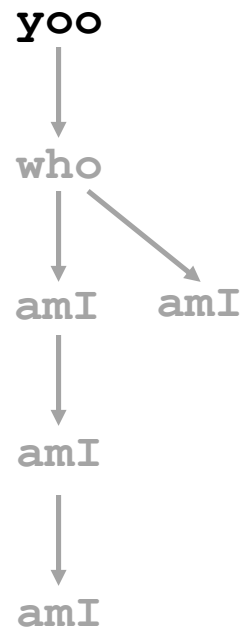


Example



Example

```
yoo (...)  
{  
  .  
  .  
  who ();  
  .  
  .  
}
```



x86-64/Linux Stack Frame

■ Caller Stack Frame

- Arguments for this call; optional
- Return address; required
 - Pushed by **call** instruction

■ Callee Stack Frame

- Saved register context
- Local variables
If can't keep in registers
- Arguments for function about to call

